NFSU Placement System: Implementation Plan

Oct 13, 2026 · @Lakshya Samay Singh

We build a Django + React + PostgreSQL web app where every placement record change needs a second approver, every sensitive access is audited, and six owners each ship one module in four phases.

Scope: compulsory vs your extra features

Build the compulsory items first (P1 to P2); your seven extras sit on top and reuse the same auth, audit and approval layers.

#	Feature	Type	Phase
1	Student database: create, read, update, archive, course/batch filter, import/export	Compulsory	P1
2	Login, RBAC, sessions, admin dashboard	Compulsory	P1
3	Policy rule engine (A+/A/B/C, I/F/I+F, switching, deadlines, absenteeism flags)	Compulsory	P2
4	Offer management (I, F, I+F, PPO, joining, NOC)	Compulsory	P2
5	Course-wise and university reports	Compulsory	P3
6	Audit log and tamper resistance	Compulsory	P1
7	Attack test cases and technical documentation	Compulsory	P3/P4
E1	Four user types: admin side, student, recruiter, public	Extra	P1
E2	Notice board (logged-in users only, admin edits)	Extra	P2
E3	Candidate chatbot (verified interview answers, own placement data)	Extra	P3
E4	Public recruiter interest form with rate limiting and email to Placement Cell	Extra	P2

#	Feature	Type	Phase
E5	Recruiter portal with ID, candidate search and resumes, talk links, attendance	Extra	P3
E6	Maker-checker: second admin or coordinator re-verifies every admin change	Extra	P1
E7	Public main dashboard: past reports, past recruiters, top packages, contact, team	Extra	P3

One naming choice: the problem statement lists four roles (Administrator, Officer, Coordinator, Recruiter). We keep all of them and add Student and Public, so your "admin" is the Placement Administrator plus Placement Officer.

Architecture and tech stack

Use a boring, battle-tested stack: Django gives parameterized ORM queries, CSRF protection, password hashing and template auto-escaping out of the box, so most secure-coding items are defaults rather than custom code.

Layer	Choice	Why / security role
Frontend	React (Vite) + TypeScript, Tailwind	Escapes output by default; never use `dangerouslySetInnerHTML`
API	Django + Django REST Framework	Serializers = server-side validation; permission classes = RBAC
Database	PostgreSQL	Constraints, row-level checks, encrypted volume, pgcrypto for sensitive columns
Cache / limits	Redis	Rate limiting, session store, OTP expiry
Auth	Django auth + Argon2id, TOTP MFA (pyotp)	Mandatory MFA for admin side, optional for students
Background jobs	Celery (or Django-Q)	Emails, deadline expiry, absenteeism flags
Email	SMTP (Gmail app password or Mailtrap for demo)	Recruiter intake, approvals, ID delivery

Layer	Choice	Why / security role
Chatbot	Retrieval over verified Q&A table + optional LLM API	See chatbot section
Files	Resumes in private storage, served via signed short-lived URLs	No public file paths
Deployment	Docker Compose, Nginx with TLS, HSTS	Encryption in transit; backups via encrypted pg_dump

Structure the backend as separate Django apps so each owner works in their own folder: `accounts`, `students`, `offers`, `policy`, `approvals`, `audit`, `recruiters`, `events`, `notices`, `chatbot`, `analytics`. Every write goes through a service function, never directly from a view, so policy checks and audit logging cannot be skipped.

Roles and permissions

Deny by default: a request is allowed only if a permission class explicitly grants it, and object-level checks stop one student reading another's record.

Capability	Public	Student	Recruiter	Coordinator	Officer	Admin
Public dashboard, contact, team	Yes	Yes	Yes	Yes	Yes	Yes
Recruiter interest form	Yes (rate limited)	n/a	n/a	n/a	n/a	n/a
Notice board (read)	No	Yes	Yes	Yes	Yes	Yes
Notice board (create/edit)	No	No	No	No	No	Yes
Own profile and own offers	No	Read only	No	No	No	No
Own resume upload	No	Yes	No	No	No	No
Student records (own course)	No	No	No	Propose and approve	Propose	Propose and approve
Mobile number, CTC details	No	Own only	Shortlisted only, masked otherwise	Own course	Yes	Yes

Capability	Public	Student	Recruiter	Coordinator	Officer	Admin
Candidate search and resumes	No	No	Yes (eligible, consenting)	No	Yes	Yes
Create and approve offers	No	No	No	Approve	Propose	Propose and approve
Verify recruiter requests	No	No	No	No	Yes	Yes
Post talk link	No	No	Yes (own events)	No	Approve	Approve
Audit log view	No	No	No	Own course	Limited	Full
Export CSV/Excel	No	No	No	Own course, logged	Logged	Logged
Chatbot	No	Yes	No	No	No	No

Two rules sit above the table. First, the same person can never propose and approve one change. Second, students have no write path to placement status, package or eligibility; those fields are absent from every student serializer, so even a tampered request cannot set them.

Data model

Offers are rows, not columns on the student, so placement history is preserved and "unique placed students" is a separate query from "total offers".

Table	Key fields	Constraints and notes
users	id, email, password_hash, role, mfa_secret (encrypted), is_active, failed_attempts, locked_until	Unique email; role is an enum
students	placement_id, user_id, name, mobile (encrypted), email, course, batch, registration_status, eligibility_status, placement_status, resume_path, archived_at	placement_id unique and immutable; course is one of the five programmes; soft delete only
companies	id, name, category (A+/A/B/C), website, verified_at	Category derived from CTC but overridable with approval

Table	Key fields	Constraints and notes
recruiter_requests	id, company details, contact, roles offered, CTC range, eligibility needs, status, reviewed_by	Public form lands here; status: new, verified, rejected
recruiters	recruiter_code, company_id, user_id, activated_at	Code is the login ID; password set via single-use link
drives	id, company_id, type (I/F/I+F), ctc_lpa, stipend, deadline, eligible_courses	One drive per company opportunity
offers	id, student_id, drive_id, offer_type, ctc_lpa, stipend_amount, stipend_period, ppo_status, acceptance, joining_status, offer_date, acceptance_deadline, source (on/off campus), noc_status	Never overwritten; new event = new row; status changes go through approvals
change_requests	id, entity, entity_id, proposed_diff (JSON), reason, proposer_id, approver_id, status, decided_at	CHECK proposer_id <> approver_id
events	id, drive_id, kind (talk, test, interview), starts_at, meeting_url (encrypted), url_approved_by	URL never returned in list APIs
attendance	event_id, student_id, joined_at, status (present, absent, excused)	Unique (event_id, student_id); absent set by job after event end
policy_flags	id, student_id, rule_code, explanation, status (open, reviewed, waived)	Output of the rule engine, human-reviewed
notices	id, title, body, posted_by, pinned, expires_at	Body sanitized on save
qa_bank	id, category, question, answer, verified_by, verified_at	Chatbot only answers from here
audit_log	id, ts, actor_id, role, action, entity, entity_id, before, after, ip, prev_hash, hash	Append-only; hash chain; DB user has INSERT only

Table	Key fields	Constraints and notes
report_snapshots	year, course, stats (JSON), published_by	Feeds the public dashboard with approved, aggregate data

Policy rule engine

The engine is a pure function: `evaluate(student, drive, offer_history, rules) -> {decision, reasons[]}` with decision one of ALLOW, BLOCK, MANUAL_REVIEW. Rules live in a config table (category thresholds, multiplier 1.5, absence window 3 drives), so the Placement Cell can tune them without code changes.

Evaluate in this order and stop at the first BLOCK, but collect all reasons for the explanation shown to staff:

1. **Registration and eligibility.** Student must be registered, eligible, and meet the drive's course/batch criteria; otherwise BLOCK.
2. **Absenteeism.** Unexcused absence in recent drives triggers a flag excluding the student from the next three drives; the flag goes to MANUAL_REVIEW so authorized staff can waive it for emergencies.
3. **Existing offer type.** Previous I blocks another I-only drive but allows F or I+F. Previous F blocks another F-only drive but allows I. Previous I+F blocks all I, F and I+F drives at other organizations.
4. **Category switching.** Compare the best existing offer with the new drive:

Existing category	New drive result
A+ (16 LPA or more)	BLOCK, student is out of the process
A (10 to under 16 LPA)	ALLOW only if the new drive is A+
B (6 to under 10 LPA)	ALLOW only if new CTC is at least 1.5 times the existing CTC
C (under 6 LPA)	ALLOW higher-category drives; confirm exact wording of the guideline for same-category moves

5. **Open acceptance.** If an earlier offer is still pending past its deadline, mark it expired; the deadline is the earlier of 24 hours or the company's stated timeline.

6. **Off-campus offers and NOC.** Off-campus offers are recorded as normal offers and run through the same checks; missing NOC where required gives MANUAL_REVIEW.

Every result is stored with its reasons (for example "BLOCK: existing category A allows switching only to A+; this drive is B"). Run the engine in two places: when staff enter an offer, and when a student applies or a recruiter shortlists, so a bypass in one UI cannot skip it. Write unit tests as a table of cases (existing offer, new drive, expected decision) so the Policy owner and Admin can verify the rules against the guideline PDF line by line.

Caveat: the guideline document was not attached to this chat, only the problem statement. Confirm edge cases (same-category C moves, boundary values like exactly 10 or 16 LPA, how I+F packages are categorized) against the actual PDF before coding the config.

Core workflows

1. Maker-checker for every admin change

1. Admin A edits a student or offer in the UI; the API does not write the record but creates a `change_requests` row holding the proposed diff and a reason.
2. A Coordinator (for that course) or a second Admin sees it in an approval queue, with a before/after view and the policy engine's verdict.
3. The approver accepts or rejects. On accept, one database transaction applies the diff and writes the audit entry; on reject, the reason is stored.
4. A database CHECK and a service-layer guard both enforce proposer is not approver. Pending changes expire after a set time.
5. Record the actor, timestamp, before and after values for propose, approve, reject and apply.

2. Recruiter onboarding

1. **Public form.** Fields: company name, website, HR contact, official email, roles, I/F/I+F, CTC and stipend range, eligible courses, expected dates, special requirements. Protect it with rate limiting (for example 5 requests per hour per IP and 3 per email per day), CAPTCHA, server-side validation, honeypot field and a size limit on text.
2. **Email to Placement Cell.** On submit, store the request and send a mail with a link to the review page. Never put raw user input into email headers; escape it in the body.

3. **Verification.** Officer or Admin checks the company, then approves or rejects; this action is audited.
4. **Approval mail.** The company gets a mail with its Recruiter ID and a single-use, time-limited link to set a password and enrol MFA. Never email a password.
5. **Recruiter portal.** Recruiter filters eligible students by course, batch, skills and CGPA range and opens resumes. Mobile numbers and emails stay masked until the Placement Cell shortlists a candidate. Every resume view is logged.

3. Talks and attendance

1. Recruiter submits a talk with date, time and meeting link through the portal; the Placement Cell approves it. The link is stored encrypted and is not included in any list API.
2. Eligible students see the talk on their dashboard. Clicking Join calls `/events/{id}/join`, which checks eligibility and time window, records `joined_at`, then redirects to the link.
3. After the event ends, a job marks every eligible student without a join record as absent and sends the result to the policy engine for the absenteeism flag.
4. Staff can mark excused absences (with reason), which goes through maker-checker.

Known limit to state in your docs: the platform proves the student clicked through, not that they stayed in the meeting. Mitigations are a short check-in code the recruiter reads out, or an attendance list uploaded by the recruiter and reconciled by staff.

4. Notice board and public dashboard

Notices are visible only after login; only Admin can create or edit, and body text is sanitized (allow-list HTML or Markdown only). The public dashboard reads only from approved `report_snapshots`: aggregate yearly stats, past recruiter names and logos, and top packages with student name shown only if the student consented. No contact numbers, emails or individual compensation are public.

Chatbot design

The chatbot answers from two controlled sources and has no direct database access, so a crafted question cannot leak another student's data.

Question type	Source	Safeguard
Interview questions and model answers	`qa_bank`, each entry approved by a faculty or alumni expert (`verified_by`, `verified_at`)	Retrieve top matches by keyword or embedding similarity; show the verified answer and its expert name; if no good match, say so instead of inventing
Placement statistics (averages, past recruiters, process info)	Pre-computed `report_snapshots` through a fixed set of server functions	Functions take no free-text SQL; return aggregates only
"My status", "my offers", "am I eligible for X"	Server functions scoped to the logged-in student's ID taken from the session, never from the message	Policy engine explains the result

Rules for the implementation:

- If you add an LLM for phrasing, give it only the retrieved snippets and a system prompt that forbids outside facts; treat user messages as untrusted (prompt-injection) and never let model output trigger actions.
- Rate limit the endpoint per user, cap message length, log questions for review, and strip personal data from stored logs.
- Provide a simple fallback (keyword search over `qa_bank`) so the demo works without an API key.
- Seed `qa_bank` with at least 40 entries across HR, technical (DSA, DBMS, networks, security) and domain topics for DFIS and AI/DS, each marked verified.

Secure coding controls and attack tests

Each control from the problem statement maps to a concrete implementation and at least one automated test; show these tests in your demo.

Area	Implementation	Test that proves it
Passwords	Argon2id, min length 12, breached-password check, lockout after 5 failures with backoff	Brute-force script gets locked; hash is never returned by any API
MFA	TOTP mandatory for Admin, Officer, Coordinator, Recruiter; optional for students	Admin login without code fails
Sessions	HttpOnly, Secure, SameSite=Lax cookies; 15 min idle timeout; rotate ID on login; server-side logout invalidation	Old session cookie rejected after logout; JS cannot read cookie
CSRF	Django CSRF tokens on every state-changing request	POST without token returns 403
SQL injection	ORM or parameterized queries only; no string-built SQL; lint rule bans raw SQL	`' OR 1=1 --` in search and login changes nothing
XSS	Output escaping, strict Content-Security-Policy, sanitize notice and remarks fields	`<script>` in notice, remarks and recruiter form renders as text
IDOR / broken access	Object-level permission on every endpoint; student queries filtered by own user ID	Student A requests `/students/B` and `/offers/B-1` and gets 404
Mass assignment	Explicit serializer field lists; student serializers exclude status, package, eligibility	Student PATCH with `placement_status` or `ctc_lpa` is ignored or rejected and logged
Switching bypass	Policy engine called in the service layer, not only the UI	Direct API call creating a blocked offer is refused with reasons
Maker-checker bypass	CHECK constraint and service guard	Admin approving own change fails; direct DB-style PATCH to a record is refused

Area	Implementation	Test that proves it
Rate limiting	Redis counters on login, recruiter form, chatbot, resume views	Burst of 20 form posts: first 5 pass, rest get 429
File upload	Allow-list PDF, size cap, random filenames, content-type sniffing, virus scan if available, private storage	`.php` renamed to `.pdf` rejected
Meeting links	Encrypted, never in list APIs, issued only through the join endpoint with eligibility check	Student not eligible gets 403; link absent from JSON lists
Email safety	Header-injection guard, escape body, SPF/DKIM if domain available	Newline in company name does not add headers
Encryption	TLS only with HSTS; encrypted DB volume; mobile and meeting URL encrypted at column level; keys from environment or secrets manager	Plain `psql` dump shows ciphertext for those columns
Audit log	Append-only table, hash chain (each row hashes previous), DB role without UPDATE/DELETE, verify command	Editing one row makes `verify_chain` report the break
Export security	Role check, reason field, row cap, watermark with actor and time, every export logged, files expire	Student or recruiter export call returns 403; log entry exists
Backup and recovery	Nightly encrypted `pg_dump`, restore script, documented restore drill	Restore to a clean container and compare row counts
Privacy and retention	Collect only listed fields; consent flag for resume sharing and for public top-package listing; retention job archives old data	Student without consent never appears to recruiters or publicly

Area	Implementation	Test that proves it
Dependencies and secrets	Pinned versions, `pip-audit` and `npm audit` in CI, no secrets in git, `.env.example` only	CI fails on high-severity advisories

Secure-coding rules for every teammate: validate on the server even if the form validated; never build SQL, HTML or shell strings from input; return generic errors to users and detailed ones only to logs; no `DEBUG=True` outside local; every new endpoint ships with a permission class and a test.

Build timeline

You did not state the hackathon length, so phases are given as shares of your total time; scale them to your deadline. If time is short, cut chatbot LLM polish and dashboard visuals before cutting any compulsory item or security test.

Phase	Share of time	Goal	Exit check
P0 Setup	5%	Repo, Docker Compose, CI, coding standards, schema v1, seed data (5 programmes, sample students, companies)	Everyone runs the app locally with one command
P1 Foundation	25%	Auth + MFA + RBAC, student CRUD, audit log, maker-checker, base UI shell	Admin A proposes, Admin B approves, audit shows both
P2 Policy and recruiting	30%	Offers I/F/I+F, rule engine, recruiter form with rate limit and mail, notice board	Blocked switch is refused with reasons; form email arrives
P3 Portals and analytics	25%	Recruiter portal, talks and attendance, reports, public dashboard, chatbot	Recruiter sees only eligible candidates; absentees auto-flagged
P4 Hardening and delivery	15%	Attack test suite, backup/restore drill, docs, demo script, bug fixing, feature freeze at start of P4	All security tests green; two full demo rehearsals

Rituals: a short sync at the start of each phase, one integration branch merged daily, no direct pushes to main, and at least one reviewer on every pull request (the Security owner reviews anything touching auth, permissions or uploads).

Team split and guidelines

Each member owns one module end to end (API, tests, a short doc page). Overview first, then the guidelines per person.

Member	Role	Owns	Hands off to
M1	Tech lead, policy engine and offers	Schema, offers, rule engine, Docker/CI, integration	Everyone (policy API)

Member	Role	Owns	Hands off to
M2	Security and identity	Auth, MFA, RBAC, sessions, rate limiting, encryption, audit log, attack tests	Everyone (permission classes, audit helper)
M3	Student records and approvals	Student CRUD, import/export, maker-checker, notice board	M1 (offers use approvals), M5 (UI)
M4	Recruiter side	Public form, email flow, recruiter portal, events, attendance	M1 (policy flags), M2 (auth)
M5	Frontend and public site	All UI screens, public dashboard, design system, accessibility	All backend owners
M6	Analytics, chatbot and QA/docs	Reports, snapshots, chatbot, test plan, technical documentation, demo script	M5 (charts), M2 (tests)

M1: Tech lead, policy engine and offers

- Freeze the schema by end of P0; every later change goes through a migration reviewed by you.
- Build `policy.evaluate()` as a pure, well-tested function with the rule table from the Policy section; keep thresholds in config.
- Implement offers as append-only events; compute placement_status from offers, never type it by hand.
- Write the deadline job (24 hours or company timeline, whichever is earlier) and the absenteeism flag job.
- Own Docker Compose, CI (lint, tests, `pip-audit`) and merge discipline.
- Done when: the case table of rules passes and a blocked switch cannot be created through any endpoint.

M2: Security and identity

- Provide reusable pieces others import: permission classes per role, an `audit.log()` helper, a rate-limit decorator, an encrypted-field type.

- Implement Argon2id, lockout, TOTP MFA, secure cookies, idle timeout, CSP and security headers, HTTPS config.
- Build the audit log as append-only with hash chain and a `verify_chain` command; revoke UPDATE/DELETE on it for the app DB user.
- Review every pull request that touches auth, permissions, uploads or exports.
- Write the attack test suite from the Security table (SQLi, XSS, IDOR, mass assignment, CSRF, brute force, upload abuse) and run it in CI.
- Done when: every row of the Security table has a passing test and a screenshot or log for the demo.

M3: Student records and approvals

- Student CRUD with unique immutable Placement ID, duplicate detection (also on email and mobile), validation, course/batch filters and search.
- Archive instead of delete; keep history. Import CSV through a validated, row-by-row preview with error report and never auto-apply.
- Build the maker-checker flow exactly as in Core workflows: proposals, approval queue, diff view, expiry, proposer is not approver.
- Route all admin-side writes (students, offers, eligibility, attendance excuses) through it.
- Build the notice board API: sanitized body, Admin-only writes, login-required reads, pinning and expiry.
- Done when: no admin-side record can change without a second person's approval, and the audit log shows each step.

M4: Recruiter side

- Public form with server validation, CAPTCHA, honeypot, rate limits and a clean thank-you page that does not reveal whether an email already exists.
- Email flow: notify Placement Cell, review page, approval mail with Recruiter ID and single-use set-password link, rejection mail.
- Recruiter portal: filters, masked contact data, resume view through signed short-lived URLs, shortlisting, all logged.
- Events: talk creation, approval, encrypted meeting link, `/join` endpoint, attendance records, end-of-event absence job, excused-absence proposal.

- Done when: a student who never used the platform link is marked absent, and the link never appears in any list response.

M5: Frontend and public site

- Build screens: public dashboard (past reports, past recruiters, top packages, contact us, team), login, student portal, admin console, approval queue, recruiter portal, notice board, chatbot widget.
- Agree on API contracts with each owner in P0 using an OpenAPI file; mock until the real API lands.
- Never render user text as HTML; keep tokens out of local storage (use the cookie session); show the policy engine's reasons clearly to staff.
- Make the public page work on mobile and meet basic accessibility (contrast, keyboard navigation, labels).
- Done when: every role can complete its main task from the UI without touching an admin tool.

M6: Analytics, chatbot and QA/docs

- Reports: totals, course-wise and batch-wise outcomes, I / F / I+F counts, company-wise offers, highest, average and median FTE CTC, with stipend reported separately from CTC, unique placed students counted separately from offers.
- Publish approved snapshots to the public dashboard; keep sensitive details out.
- Chatbot as described in its section, seeded with 40+ verified Q&A and the fallback keyword search.
- Maintain the test plan, the technical document (schema, architecture, security controls, tests, limitations) and the demo script; collect evidence (logs, screenshots) from M2's tests.
- Done when: report numbers match a hand calculation on the seed data, and the documentation covers every deliverable in the problem statement.

Definition of done, demo and limitations

The project is done when a judge can watch one story end to end and see each security claim proven live.

Demo script (about 8 minutes)

1. A recruiter submits the public form; rate limiting blocks the sixth attempt; the Placement Cell receives the mail.
2. Officer verifies it; the recruiter gets an ID, sets a password and MFA, and posts a talk link.
3. Student joins through the platform link and is marked present; a second student who skips it is marked absent and flagged.
4. Admin A enters an A-category offer for a student; Admin B approves it; audit log shows both steps.
5. The same student is blocked from a B drive with the policy engine's written reason; an A+ drive is allowed.
6. Student tampers with a request (own package, another student's ID) and fails; the attempt appears in the audit log.
7. Show the public dashboard, notice board, chatbot answer with expert name, and a report where unique placed students differ from total offers.
8. Edit one audit row in the database and show `verify_chain` detecting it.

Known limitations to state honestly

- Attendance proves a click-through, not continued presence in the call.
- The policy config follows the guideline text we could read; confirm the exact PDF wording for edge cases before final submission.
- Email depends on the SMTP service available during the hackathon; use a sandbox mailbox for the demo.
- The prototype uses seed or anonymized data; real student data needs institutional approval and a hardened production deployment.

Open questions for you

- What is the hackathon deadline and the preferred language or framework of your team? The plan assumes Python/Django and React; if your team is stronger in Node or Java, the module split still applies.
- Can you share the Student Placement Guidelines 2026-27 PDF so M1 can turn it into the rule table exactly?